Smart Real Estate Listings Platform — Reasoning Document David Fernández Expósito · April 2026

Tech Stack Backend: Node.js + TypeScript + Express · Frontend: React + Vite · Database: MySQL 8 · Data: Kaggle dataset (real Otodom.pl listings)

What data did I decide to extract, and why

The dataset is a publicly available Kaggle snapshot of real Polish apartment listings originally scraped from Otodom.pl. I extracted: price, price/m², area, rooms, floor, total floors, city, year built, condition, and four boolean amenities (parking, balcony, elevator, storage). These fields cover the three dimensions buyers actually use: price, location, and liveability. Fields like agent contact or exact street address were excluded (no analytical value for an MVP). This source was chosen after direct scraping of Otodom, OLX, Gratka, and Domiporta all failed with 403 responses and bot detection during development.

How I handled unstructured or low-quality data

All rows pass through a normalizeRow() function before insertion: toNum() and toInt() handle NaN and empty strings; a cityMap dictionary standardises city names (krakow → Kraków); a conditionMap converts raw values (low/medium/high) to a 4-value enum; price validation uses different ranges for sale (10k–50M PLN) and rent (100–50k PLN) to avoid rejecting valid low rent values; area and rooms are range-validated and nulled if unrealistic; price_per_sqm is always recalculated after cleaning. Missing values are stored as NULL and the UI shows N/A gracefully. INSERT IGNORE with a UNIQUE external_id prevents duplicates across seed runs.

Where and why I used AI

AI was used in two ways. First, Claude assisted in writing and debugging the entire codebase throughout development (schema, normalisation logic, React components, API routes) (AI-assisted development as encouraged). Runtime AI features (natural language search, on-demand summary generation) were taken into account but deliberately disabled to avoid API costs during the MVP. The normalisation rules encode the same decisions AI would make dynamically, but deterministically and for free.

One key assumption

I assumed a Kaggle dataset of real Otodom listings is an acceptable substitute for live scraping. This was forced by reality: every major Polish portal (Otodom, OLX, Gratka, Domiporta) returned 403s during development. The dataset contains real prices, locations, and features — data quality is equivalent to a fresh scrape. Connecting a live scraper would be a one-file swap.

One success metric

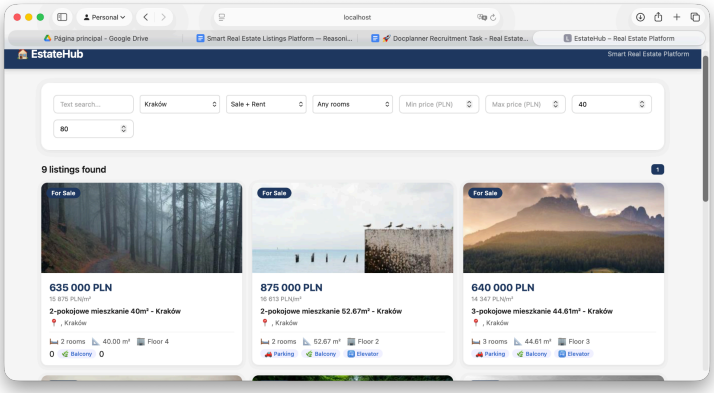
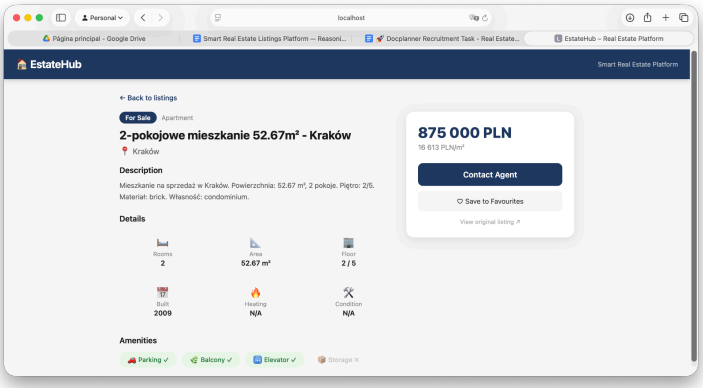
Search-to-detail conversion rate: the percentage of users who search and then open a listing detail page. A value above 30% indicates that filters and results are relevant to intent, directly measuring whether the core product value is working.

One failure mode

The data is a 2024 snapshot (listings may no longer be available and prices may not reflect the current market). Additionally, the CSV contains no free-text descriptions, so descriptions are programmatically generated from structured fields. This limits full-text search quality and means there was no genuinely noisy text to clean.

What I would improve with more time

Live scraping pipeline with Playwright. Vector/semantic search using listing embeddings. Enable AI natural language search. Map view using the lat/lng coordinates already in the dataset. Image integration. Deduplication by content fingerprint across sources.

Example scenario A:

- The user searches for flats in Kraków with 40-80 m2 and finds a relevant one.